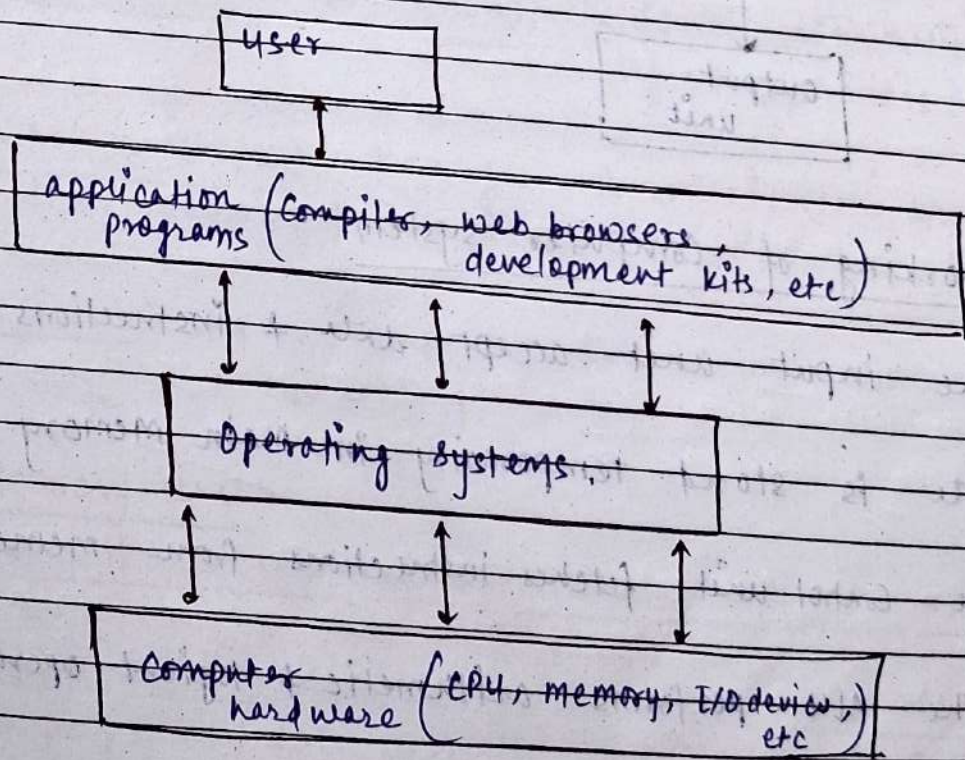


UNIT-1

Introduction to Operating System (OS)

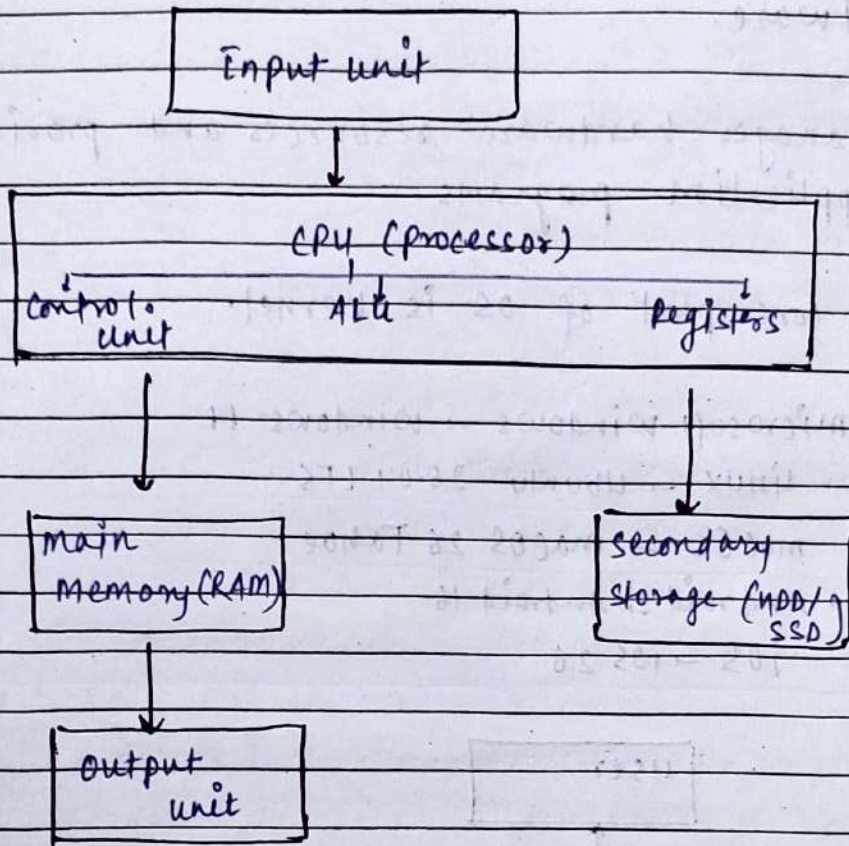
- * An operating system is system software that acts as an interface b/w the user and computer hardware.
- * It manages hardware resources and provides services for application programs.
- * The core part of OS is kernel.

Ex: Microsoft Windows → Windows 11
Linux → Ubuntu 20.04 LTS
macOS → macOS 10.15
Android → Android 10
iOS → iOS 13



Computer System organization

Computer system organization refers to the way the hardware components of a computer are arranged and interconnected to perform data processing and execute instructions efficiently.



Working of Computer system

- (i) The input unit accept data & instructions.
- (ii) Data is stored temporarily in Main Memory (RAM).
- (iii) The Control unit fetches instructions from memory.
- (iv) The ALU performs arithmetic & logical operations.

(v) Results are stored back in memory.

Page No. _____

Date _____



(vi) The output unit displays the processed information.

(vii) Data can be saved permanently in secondary storage.

Components of a Computer System

(i) Input unit: Accepts data & instructions from the user and sends to the CPU in machine-readable form.

Ex: Keyboard, Mouse, Scanner.

(ii) Central Processing Unit: Also known as brain of the computer because it performs all calculations & control system operations.

It consists of:

* Arithmetic logic unit (ALU): Perform arithmetic & logical operations.

* Control unit (CU): Manages communication b/w CPU, memory & I/O devices.

* Registers: Stores temporary data & instructions during processing.

(iii) Memory Unit:

Primary memory

* Stores programs & data currently being used.

Ex: RAM, ROM, Cache memory.

Secondary memory

* Stores data permanently. Larger capacity but slower.

Ex: Hard Disk, USB Drive, DVD

(iv) Output unit :- Receives processed data from the CPU.

Page No. _____

Date _____

* Converts it into human-readable format.

* Ex:- Monitor, Speakers, Printer, Projector.

Computer System Architecture :-

* Computer system architecture is the conceptual design and organisation of a computer system.

* It defines how the CPU, Memory, I/O devices & storage are connected and interact to execute instructions and process data.

Difference b/w Computer System Organization and Computer System Architecture

Computer System Architecture

* Define the design & structure of a computer system.

* Describes what the computer does.

* Concerns with logical design of the system.

* Includes instruction sets, data formats, addressing modes & CPU design.

Computer System Organisation

* Defines the implementation & arrangement of hardware components.

* Describes how the computer perform those tasks.

* Concerns with physical implementation of the system.

* Includes control signals, memory organization & hardware connections.

Operating structure

Operating system structure

* Operating system structure refers to the internal design and organization of an operating system.

* It defines how different components of the OS are arranged and interact to provide services to users and manage computer hardware.

Types of OS structures:-

(i) Simple structure:- The entire OS works as a single large program.

* All system services are placed together without clear separation.

* Easy to develop but difficult to maintain.

Ex: MS-DOS

(ii) Monolithic structure:- The OS consists of one large kernel containing all services.

* All components run in kernel mode and can communicate easily.

Ex: UNIX, LINUX

(iii) Layered structure:- The OS is divided into several layers. Each layer performs a specific function.

Ex: THE operating system.

(iv) Microkernel structure: Only essential functions (CPU scheduling, memory management, I/O remain in the kernel.

* Other services run in user space.

Ex: Minix, QNX

(v) Modular structure: In this kernel is divided into separate modules. Modules can be loaded or unloaded as needed.

Ex: Modern Linux

(vi) Hybrid structure: Combines features of monolithic and microkernel architectures.

Ex: Microsoft Windows, Apple MacOS.

Goals of Operating System

(i) Primary goal: To make system user friendly.

(ii) Secondary goal: To make system efficient.

(iii) Role → User view: Design to maximise the work that the user is performing.
→ System view: In this program work as a resource.

Operating System Operations / functions

(i) Process Management:

* Manages the creation, execution, scheduling and termination of processes.

* Allocates CPU time to different processes.

* Supports multitasking and multiprocessing.

Functions:

- * Process creation & deletion
- * CPU scheduling
- * Process Synchronization
- * Deadlock handling.

(ii) Memory Management:

* Manages the computer's main memory (RAM).

* Allocates and deallocates memory to processes

* Keep track of memory usage.

Functions:

- * Memory allocation
- * Memory protection
- * Virtual memory management
- * Swapping.

(iii) User Interface (UI): Provides a way for user to interact with OS.

Types:

Command line Interface (CLI): Users type command.

Graphical User Interface (GUI): Users interact through windows, icons, menus & pointers.

(iv) File system management:

- * Organizes & manages files and directories
- * Controls file access & storage.

Functions:-

- * Create and delete files
- * Read and write files
- * File permissions
- * Backup and recovery.

(v) Device (I/O) Management:

- * Controls and manages I/O devices using device drivers.
- * Allocates devices to processes.

Functions:-

- * Device allocation.
- * Buffering
- * Caching
- * Spooling.

(vi) Secondary storage Management:

- * Manages hard disk, SSDs, and other storage devices.
- * Organizes data for efficient storage and retrieval.

Functions:-

- * Disk scheduling
- * Free Space Management
- * Disk allocation.

(vii) Error Detection and Handling:

- * Detects software & hardware errors.
- * Maintains system stability by reporting and recovering from errors.

(viii) Security and Protection:

- * Protects the system from unauthorized access.
- * Ensures data confidentiality & integrity.

Functions:-

- * User authentication.
- * Access Control
- * Password management
- * Data protection.

(ix) Networking:

- * Enables communication b/w computers over a network.
- * Supports resource and data sharing.

Functions:-

- * Network communication
- * File sharing
- * Remote access
- * Internet connectivity.

(x) Command Interpreter (Shell):

- * Acts as an interface b/w the user & the operating system.
- * Accept user commands & executes them.

Examples:-

- * Bash (Linux)
- * Powershell (Windows)
- * Command Prompt (Windows).

Types of operating system

(i) Batch operating system: A batch OS executes a group (batch) of jobs together without any user interaction during execution.

* Efficient for large-volume jobs.

* Suitable for repetitive tasks.

Ex: IBM OS/360

(ii) Time-sharing operating system: It allows user to run multiple applications simultaneously by sharing CPU time among them.

* CPU executes several tasks by rapidly switching between them.

* Provides quick response to users.

Ex: Microsoft Windows, macOS.

(iii) Distributed operating system: It manages a group of interconnected computers and make them appear as a single system.

* High reliability.

* Faster computation

Ex: Amoeba, Plan 9

(iv) Network operating system: It manages resources & enable communication b/w connected computers.

* Easy resource sharing & improved security.

Ex: Windows Server, Linux server.

(v) Real Time operating system: It processes data & response within a fixed time limit.

* Fast and predictable response.

Ex: GNU, FreeRTOS.

(vi) Embedded operating system: It is a specialized OS designed to run on embedded systems - computers built into devices to perform specific tasks.

* It is optimized for small size, high reliability, low power consumption, and real-time performance.

Ex: VxWorks, GNU

(vii) Mobile operating system: A mobile operating system is designed specially for smartphones, tablets and other portable devices.

* Wireless communication support

* Portable & user-friendly.

Ex: Android, iOS.

(viii) Multuser operating system: It allows multiple users to access the same computer simultaneously.

- * Each user gets separate resources & permissions.
- * Supports multiple users' accounts.

Ex: UNIX, Linux

(ix) Multitasking operating system: It is an operating system that allows multiple programs or tasks to run simultaneously by rapidly switching the CPU among them.

* Ex: Linux, macOS, iOS.

(x) Desktop operating system: It is an operating system for personal computers and laptops.

- * It provides a user-friendly interface, manages hardware resources, and allows user to run multiple applications efficiently.

Ex: macOS, Microsoft Windows, Ubuntu.

(xi) Multiprogramming operating system: It keeps multiple programs in memory at the same time and executes them by switching the CPU among them.

- * CPU switches to another job when one waits for I/O.
- * Improves CPU utilization.

Ex: Linux, Unix

Kernel

* It is the central and most important part of an operating system.

* It acts as an interface b/w application programs and hardware.

* It runs in privileged mode and manages important system resources such as:

- CPU
- Main Memory
- Files
- I/O Devices
- Security and Protection.

Functions of kernel

* Process management: Creates, schedules & terminates process.

* Memory Management: Allocates & deallocates RAM.

* File Management: Controls creation, reading & writing of files.

* Device Management: Manages hardware devices through device drivers.

* I/O Management: Controls I/O operations.

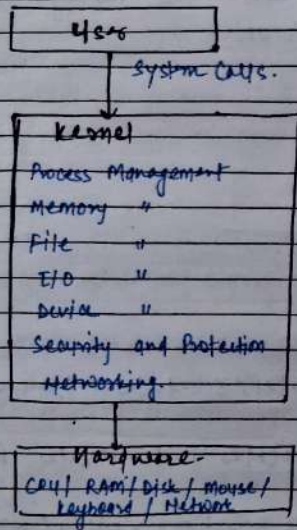
* Security: Provides protection and access controls.

* Networking: Handles communication b/w computers.

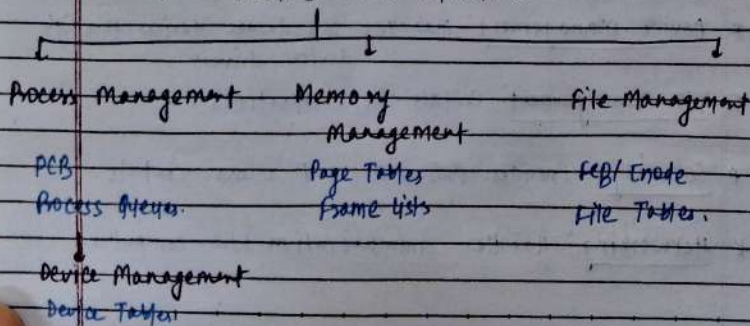
Kernel Data structures

* Kernel data structures are data structures maintained by the operating system kernel to store & manage information about system resources.

* They help the kernel efficiently manage processes, memory, files, devices and other resources.



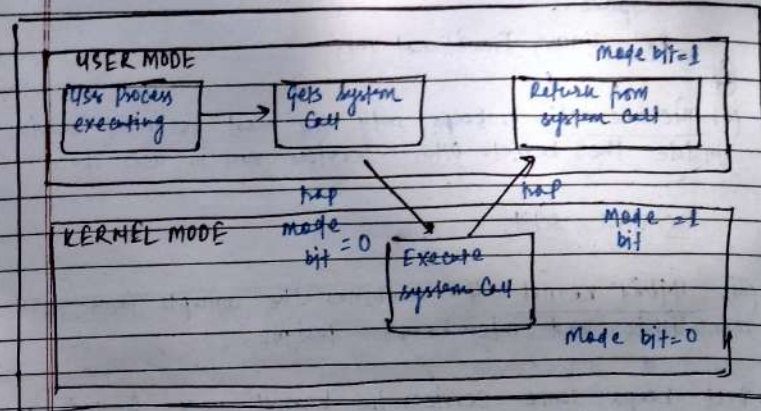
KERNEL Data structure



* User Mode v/s kernel mode

OS works in two modes

- User Mode (eg. Text editor)
- kernel mode



User Mode

* Also called unprivileged, restricted or slave mode

* Mode bit = 1

* Cannot directly access kernel program

* Applications are ^{less} more privileged compared to kernel mode

* Ex: Browser, games, editor

Kernel Mode

* Also called master mode, privileged or system mode

* Mode bit = 0

* Access both user and kernel programs

* Applications are more privileged compared to user mode

* Ex: memory manager, scheduler

Types of kernel

(i) Monolithic kernel:- A monolithic kernel contains most operating system services inside the kernel space.

Ex:- Linux, Traditional Unix

(ii) Microkernel:- It keeps only the most essential services inside the kernel. Other services run in user space.

Ex:- QNX, sel4

(iii) Hybrid kernel:- It combines the concepts from monolithic and microkernel designs.

* It keeps some services in kernel space for performance while using a more modular architecture.

Ex:- Apple XNU, Windows NT family.

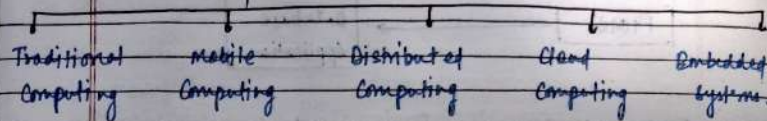
(iv) Modular kernel:- It has a core kernel with additional functionality that can be loaded or removed as modules when required.

Ex:- Modern Linux.

Computing Environment

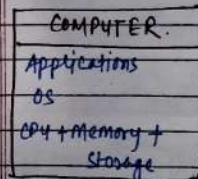
* It refers to the combination of hardware, software, network and services in which computer programs execute and users perform computing tasks.

Types of Computing Environment



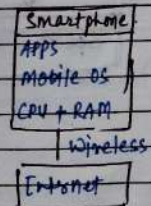
* Traditional Computing Environment

In this a computer has its own CPU, memory, storage and operating system.



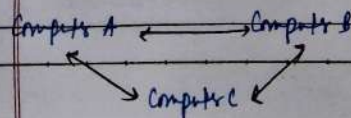
* Mobile Computing Environment

It uses mobile portable devices such as smartphones & tablets.



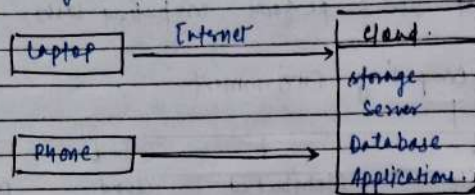
* Distributed Computer Environment

In this multiple computers work together and communicate through a network.



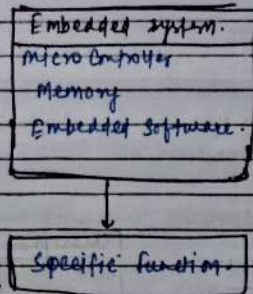
* Cloud Computing Environment

It provides computing resources such as storage, processing power and applications over the internet.



* Embedded Computing Environment

It is a computer system designed to perform a specific function within a larger device.



Open Source Operating System

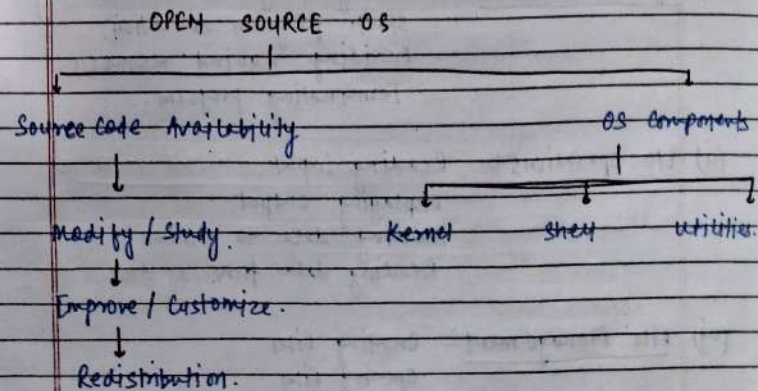
Open source operating system is an OS whose source code is publicly available. Users and developers can study, modify, improve and redistribute the source code according to its license.

Features:-

- * Source code availability
- * Modification
- * Redistribution
- * Community development
- * Customization.
- * Security and Transparency.

} Advantages

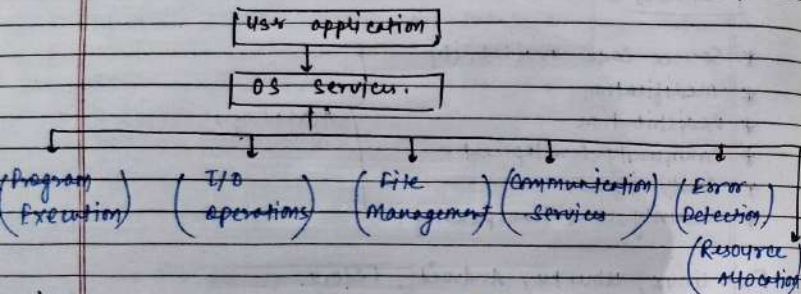
Ex: Linux, Ubuntu, Android, Fedora.



Operating System Services.

* An OS provides a set of services to user and application programs.

* These services make it easier to use the computer and allow programs to execute safely and efficiently.



(i) Program Execution:- Loading programs into memory
Starts program execution.
Providing required resources.
Terminating program.

(ii) I/O operations:- Reading input
Displaying output
Sending data to printer
Reading data from a disk.

(iii) File Management:- Creating files
Opening files
Reading/Writing files
Deleting files
Renaming files.

(iv) Communication services:- The OS provides mechanism that allows processes to exchange information.

(v) Error Detection:- OS detects error
Takes appropriate actions
System Continuous Safety.

(vi) Resource Allocation:- A computer has limited resources so the OS must allocate them among different processes.
Resources include CPU, Main memory, storage, I/O devices.

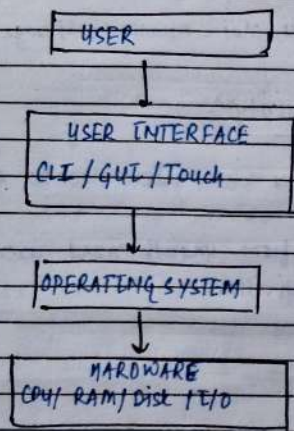
(vii) Protection and Security:-

- * Controls how processes and users access system resources.
- * Protects the system from unauthorized users and malicious activities.

Users and operating System Interface

* It is the method through which a user communicates with and control operating system.

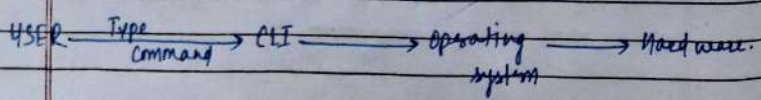
* The O.S provides an interface so that users can execute programs, manage files, access devices and perform other operations without directly interacting with hardware.



Types of user interfaces associated with OS:-

(i) Command line Interface (CLI): It allows users to interact with the operating system by typing command.

Ex: Windows Command Prompt

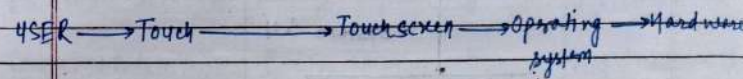


(ii) Graphical user interface: It allows users to interact with the O.S using graphical elements.

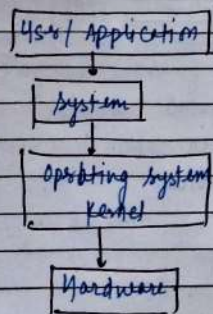
- * It includes: Windows
- Icons
- Menus
- Buttons
- Pointers.

(iii) Touch Based Interface: It allows users to interact with O.S by touching the screen.

- * It is commonly used in: Smartphones
- Tablets
- ATMs
- Smart Devices.



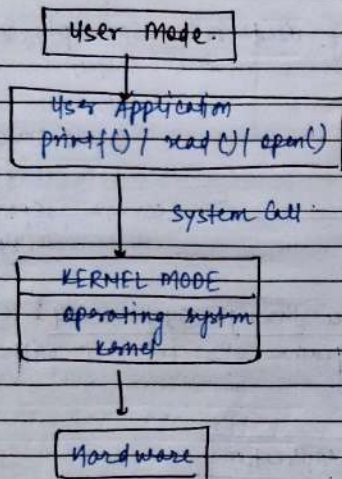
(iv) User Interface through System Call: Application Programs also interact with the O.S using system calls.



System Call

* A system call is a mechanism through which a user program requests a service from the operating system kernel.

* User programs normally run in user mode, where they have limited privileges. When they need an operation that requires OS level access - such as reading a file, creating a process - they make a system call.



How System Call works:-

- (i) User program requests a service.
- (ii) system call is generated
- (iii) CPU switches from user mode to kernel mode.

(iv) Kernel performs the requested operation.

(v) Result is returned to the program.

(vi) CPU switches back to user mode.

Types of System Calls (P-F-D-E-C-P)

Types	Purpose	Examples
Process Control	Manages Process	fork(), exit(), exec()
File management	Manages files	open(), read()
Device Management	Manage E/O device	write(), read()
Information Maintenance	Manages system information	getpid(), time()
Communication	Process Communication	send(), receive()
Protection	Manage access permissions	chmod(), chown(), access()

System Program

* System Programs are programs provided by the operating system that offer a convenient environment for users to develop, execute and manage programs.

* They act as a layer b/w the user/application programs and the operating system services.

Types of system Program

(i) File Management: Used to create, delete, copy, rename, print & manipulate files and directories.

Ex: copy, delete, mkdir, rmdir.

(ii) Status Information: These programs provide information about the system and its resources.

* They can display: Date and Time
System Configuration
Network information
Memory usage.

(iii) File Modification: These programs ^{are} used to modify the contents of files.

Ex: Text editors
Search utilities
Sorting programs
File-formatting utilities.

(iv) Programming language Support: They provide tools that help programmers write, translate and execute programs.

* These includes: Editor
Compiler
Assembler
Interpreters.

(v) Program loading and Execution: These help ^{users} load and execute programs.

Program → Load → Main Memory → CPU → Execution.

(vi) Communication: These programs provide mechanisms for communication b/w users, processes and computers.

Ex: Email programs, Web browsers, File-transfer utilities.

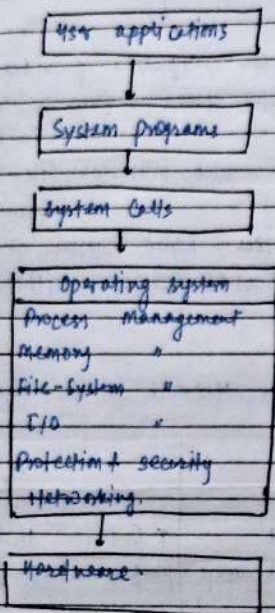
(vii) Background Services: Some system program runs in background and provide services without direct user interactions.

Ex: Printing Services, Network services, Logging Services.

System Program	System Calls
* Programs that provide a convenient environment for users.	* Interface through which programs request kernel services.
* Runs as software programs.	* Enter the OS kernel.
* Usually operate in user space.	* Execute kernel services.
* <u>Ex</u> : Text editor Compilers File utility.	* <u>Ex</u> : open() read() write()
* Enables for users to interact with.	* Mainly used by programs.

Operating System Design

Operating system design determines the goals, structure, components, and interaction of the operating system.



Goals of Operating System Design

(i) User Goals:-

- * Easy to use
- * Convenient
- * Reliable
- * Secure
- * Responsive

(ii) System Goals:-

- * Use hardware efficiently
- * Provide good performance
- * Be maintainable
- * Be flexible
- * Be scalable
- * Reliable & secure

Important Design Principles

- Modularity
- Portability
- Reliability
- Security
- Efficiency

Operating System Implementation

After designing the OS, the next step is implementation which means writing the actual software.

Implementation Process

Requirements

↓

Design

↓

Choose OS structure

↓

Select programming language

↓

Implement kernel components

↓

Testing & Debugging

↓

Optimization

↓

Deployment

↓

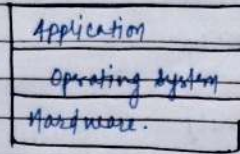
Maintenance and updates.

Operating system structures

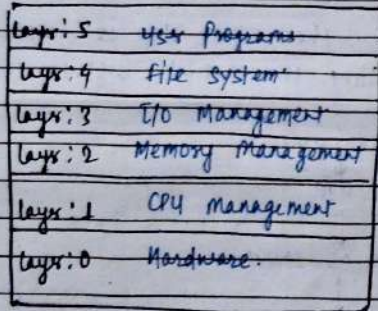
It refers to the different approaches that can be used to organise an operating system.

(i) Simple structure: There is a little separation b/w components.

* Simple to design.



(ii) Layered structure: The OS is divided into several layers. Each layer can use services provided by the layer below it.



* Easier debugging

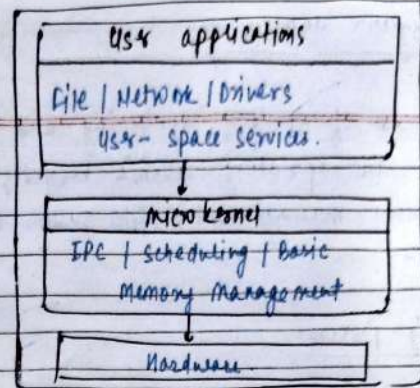
* Better modularity.

(iii) Microkernel structure: Only the most essential functions are kept inside the kernel.

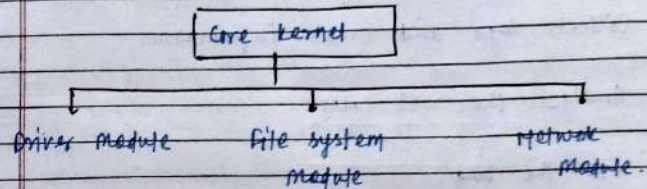
* Better isolation

* Easier to extend

* Improved Reliability.



(iv) Modular structure: The OS structure contains a core kernel and dynamic loadable modules.



(v) Hybrid structure:

It combines features of different kernel structures.

Ex: Windows NT architecture, Apple's XNU.

Operating system debugging

It is the process of finding, analyzing and fixing errors (bugs) in an operating system's kernel, device drivers, memory management, file system, or other core components.

OS debugging Process

- (i) Identify the problem.
- (ii) Reproduce the issue.
- (iii) Collects logs and crash information.
- (iv) Analyze the root cause.
- (v) Fix the bug.
- (vi) Test the fix.
- (vii) Verify that the issue is resolved.

Advantages

- * Improves system stability and reliability.
- * Enhances security by identifying vulnerabilities.
- * Prevents system crashes and data loss.
- * Ensures correct interaction b/w hardware and software.

Operating System Generation

It is a process of preparing and configuring an operating system so that it can run on a particular computer system or hardware configuration.

Steps in operating system generation

- (i) Hardware Analysis:- Identify the available hardware such as CPU, mouse, keyboard, network devices.
- (ii) System Configuration:- Select the required OS components and hardware-specific options.
- (iii) Kernel Configuration:- Configure the kernel according to the processor, memory, file system and other hardware requirements.
- (iv) Device Driver Selection:- Select and include drivers required for devices such as graphic cards, printers, storage devices and network adapters.
- (v) Building the kernel:- Compile the selected OS components and create the operating system kernel.
- (vi) Installation:- Install the generated OS on the target computer.
- (vii) Testing:- Test the OS to ensure that hardware and software components work correctly.

Types of operation system generation.

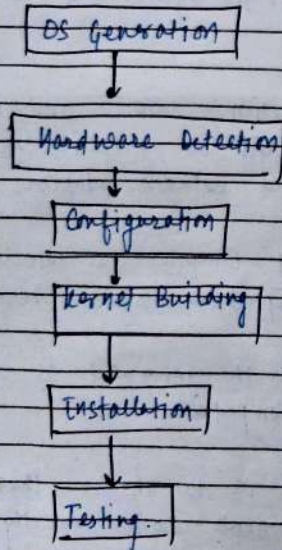
Page No. _____
Date _____

(i) Manual Generation

- * System administrator manually selects and configure OS components.
- * Provides more control over the system.

(ii) Automatic Generation

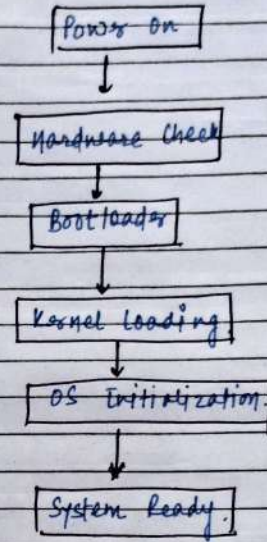
- * Tools automatically detect hardware and configure the OS.
- * Saves time and reduces configuration errors.



System Boot

It is the process of starting a computer and loading the operating system into the main memory (RAM) so that the computer becomes ready for use.

System Boot Process



Types of system boot

- Cold Boot: Starting a computer from a completely powered-off state.
- Warm Boot: Restarting the computer without completely turning off the power.

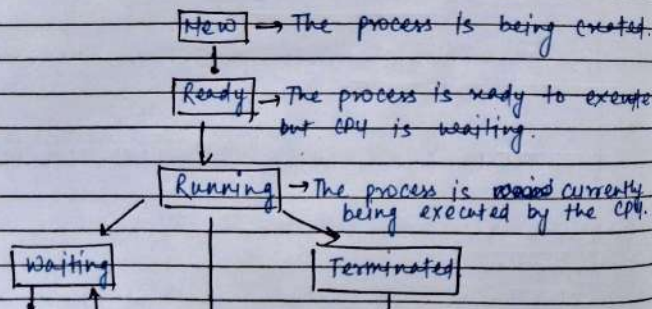
Process Concept in Operating System.

A process is a program in execution. It is an active entity that requires CPU, memory, files, and other resources to perform its tasks.

Components of a Process.

- (i) Text Section: Contains the program code or instructions.
- (ii) Data Section: Contains global and static variables.
- (iii) Heap: Used for dynamically allocated memory.
- (iv) Stack: Store function calls, local variables and parameters.

Process States

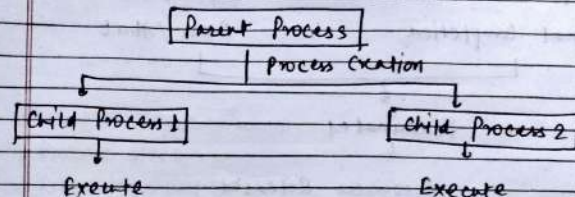


The process is waiting for an event, usually an I/O operation.

The process is completely executed or has been stopped.

Process Creation:

- * It is the process by which the operating system creates a new process for execution.
- * Ex: `fork()` → creates a new process.
- * A process that creates another process is called parent process. While the created process is called child process.

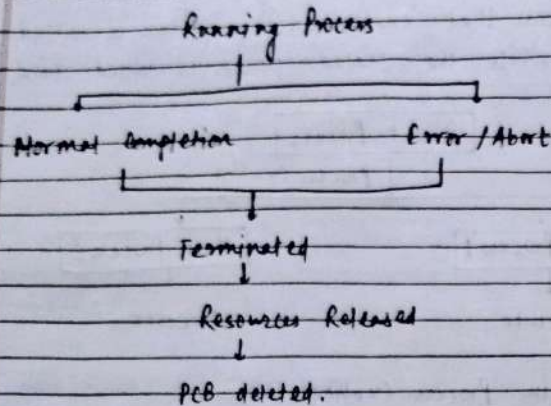


Steps in Process Creation:

- (i) Assign a Process ID: The OS assigns a unique PID for a new process.
- (ii) Create PCB: The OS creates process control block (PCB) for the new process.
- (iii) Allocate Resources: Memory, CPU, files and other required resources are allocated.
- (iv) Load Program: The program code and data are loaded in the memory.
- (v) Initialize Process: The OS initializes the program counter, CPU registers, process state and other information.
- (vi) Place in ready queue: The newly created process is placed in ready queue, where it waits for CPU allocation.

Process Termination:-

It occurs when a process finishes its execution or is stopped by the operating system or other process.



Steps during Termination:-

- * The process stops executing.
- * The OS releases the resources allocated to it.
- * Open files and I/O resources are also released.
- * Memory allocated to the process is freed.
- * The process information is removed from scheduling structure.
- * The PCB is eventually removed.

Process Control Block (PCB)

A PCB is an OS data structure that stores information about a process, including its PID, state, program, counts, CPU registers, scheduling, information, memory information and I/O status.

It contains:-

- * Process ID
- * Process state
- * Program counts
- * CPU registers.
- * CPU scheduling information.
- * Memory management information.
- * I/O status information.
- * Accounting Information.

Process Scheduling

It is the activity of the operating system that selects a process from the ready queue and assigns to the CPU.

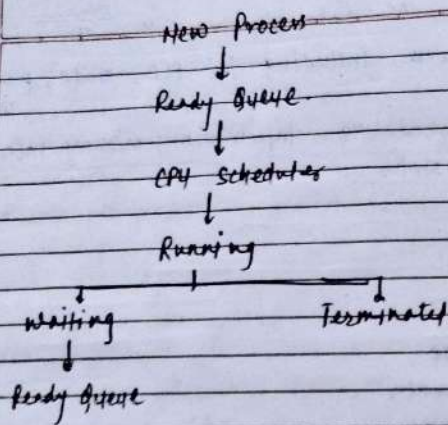
Main Purpose: * Keep the CPU busy.

- * Improves CPU utilization.
- * Reduce waiting time
- * Provide quick response to the processes.

Process Scheduling Diagram

Page No.
Date

Ajanta



Operations on Processes

The operating system performs several operations to manage processes throughout their lifetime.

(i) Process Creation:- The OS creates a new process when a program needs to execute.

* The new process is assigned a PID and a PCB.

(ii) Process Scheduling:- The OS selects a process from the ready queue and gives it to the CPU.

* Scheduling helps in efficient CPU utilization.

(iii) Process Execution:- The selected process executes its instructions using the CPU.

* It may continue running or move to a waiting state for I/O.

(iv) Process Blocking / Waiting:- A process enters the waiting state when it needs to wait for an event, such as I/O completion.

(v) Process Resumption:- A waiting process can be brought back to the ready state.

* The scheduler may later assign the CPU to do it again.

(vi) Process Suspension:- The OS may temporarily stop a process and remove it from active execution.

(vii) Process Termination:- A process is terminated when it completes execution or encounters an error.

* The OS releases its memory, files and other resources.

Interprocess Communication (IPC)

It is a mechanism provided by the operating system that allows two or more processes to communicate and exchange data with each other.

Two main IPC models

(i) Shared Memory:- In shared memory, the OS creates a memory region that can be accessed by multiple processes.

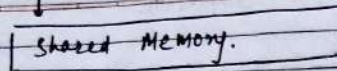
* Very fast communication.

* Suitable for exchanging large amount of data.

* Requires synchronization to prevent processes from accessing shared data incorrectly.

Process A

Process B

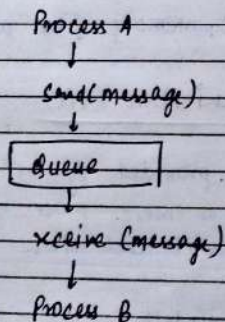


(ii) Message Passing:- In message passing, processes communicate by sending and receiving messages through the operating system.

* Easy to implement.

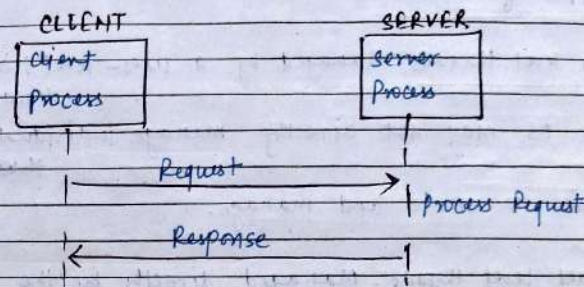
* Useful when processes are running on different systems.

* Usually slower than shared memory because communication involves system call.



Communication in Client-Server Systems

Communication in client-server system is the process through which a client sends a request to a server and the server processes the request and sends a response back to the client.



How communication works

- * Client sends a request
- * Server receives the request
- * Server processes the request
- * Server sends a response.
- * Client receives the response.

Threads in operating system

A Thread is the smallest unit of CPU execution within a process. A process can contain one or more threads that can execute independently.

Types of threads

(i) User level threads: Managed by a user-level thread library.

- * The OS may not directly manage individual threads.
- * Faster to create and manage.

(ii) Kernel level threads: Managed directly by the operating system kernel.

- * The OS schedules these threads.
- * Better support for parallel execution on multiple CPU cores.

Process

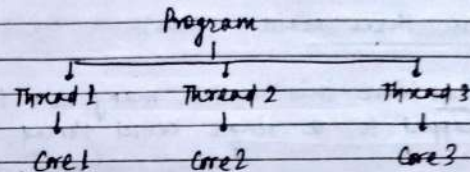
Thread

- | Process | Thread |
|---|--|
| * Independent program in execution | * Smallest execution unit within a process. |
| * Has its own memory space. | * Shares memory with other threads of the process. |
| * Execution is relatively expensive. | * Creation is relatively faster. |
| * Communication is comparatively complex. | * Communication is easier. |
| * Context switching is more expensive. | * Context switching is generally cheaper. |

Multicore programming

* It is a technique in which a program is designed to use multiple CPU cores simultaneously to perform tasks faster and more efficiently.

* A large program is divided into smaller tasks or threads. These threads can execute on different CPU cores at the same time.



- Advantages:
- * Faster execution
 - * Better ~~executed~~ performance.
 - * Increased throughput
 - * Better responsiveness.
 - * Efficient Resource utilization.

Concurrency v/s Parallelism

Concurrency	Parallelism
* Multiple tasks make progress. * Can occur on a single core. * Focuses on managing tasks.	* Multiple tasks execute at the same time. * Requires multiple cores for true simultaneous execution. * Focuses on executing tasks simultaneously.

Multithreading:- A process of containing multiple threads is called multithreaded process.

Multithreading Models

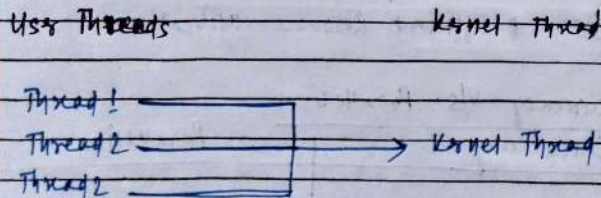
Multithreading models describe how user-level threads are mapped to kernel level threads in an operating system.

There are three main models:-

(i) Many to One:- In this model, many user threads are mapped to a single kernel thread.

* Simple to implement.

* Thread management is fast because it is done at the user level.



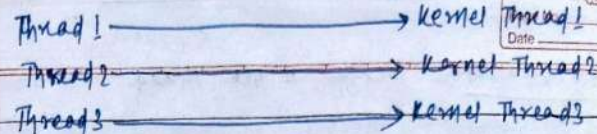
(ii) One to One:- In this model, each user thread is mapped to one kernel thread.

* If one thread blocks, other threads can continue.

* Allows true parallelism on multicore systems.

User Threads

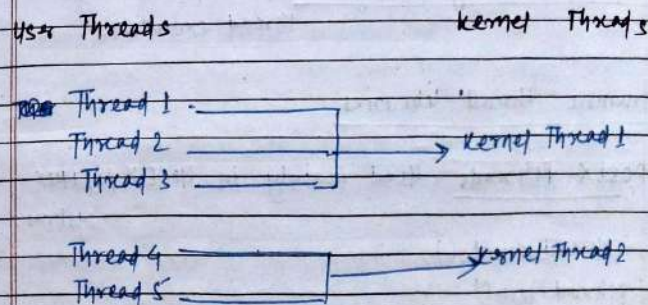
Kernel Threads



(iii) Many to Many Model:- In this model, many user threads are mapped to a smaller or equal number of kernel threads.

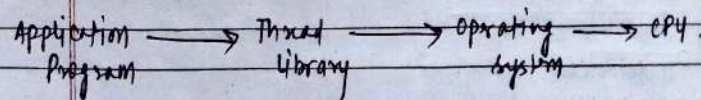
* Supports many user threads.

* If one thread blocks, other threads can continue.



Thread Libraries

A thread library is a collection of functions or classes provided by a programming language or operating system to manage, create, synchronize and terminate threads.



Main Functions of Thread Libraries

- (i) Thread Creation:- Creates a new thread.
- (ii) Thread Termination:- Ends a thread.
- (iii) Thread joining:- Makes one thread wait for another thread to finish.
- (iv) Thread Synchronization:- Control access to shared resources.
- (v) Thread Scheduling/Management:- Helps manage thread execution.

Common Thread Libraries

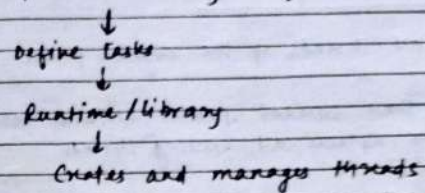
- (i) POSIX Threads:- Used mainly in UNIX/LINUX systems.
`pthread_create()`
`pthread_join()`
`pthread_exit()`
- (ii) Windows Thread Library:- Provided by Windows operating system.
- (iii) C++ Thread Library:- C++ introduced the standard `<thread>` library.
`std::thread`
`join()`
`detach()`
`get_id()`

Implicit Threading

It is a technique in which the programmer does not explicitly create and manage thread. Instead, the programming environment, compiler or runtime system automatically manages the thread.

It makes multithreaded programming easier and reduces the complexity of thread management.

Implicit Threading Program



Advantages

- * Easier to write multithreaded programs.
- * Reduces thread-management complexity.
- * Improves program reliability.
- * Useful for multicore processors.

Disadvantages

- * Give the programmer less control over individual threads.
- * Debugging can sometimes be difficult.

Threading Issues

Threading issues are problems that can occur when multiple threads execute concurrently and share resources. These issues can cause incorrect results, crashes or poor performance.

Major Threading Issues

(i) fork() and exec() System Call:

fork() → Duplicate all threads of the original process.

exec() → Replaces the current process with a new program and normally replaces all existing threads.

(ii) Signal handling:- Signals are notifications sent to a process or thread to indicate an event.

In a multithreaded process, it is necessary to determine which thread should handle the signal.

(iii) Thread Cancellation:- It means terminating a thread before it finishes execution.

(iv) Thread-local Storage (TLS):- Thread-local storage provides separate data for each thread.

(v) Scheduler Activations:- It provides communication between the user-level ~~library~~ thread library and the operating system kernel.

Other Common Threading Problems

(i) Race Condition:- Two or more threads access shared data at the same time, causing an incorrect result.

(ii) Deadlock:- Two or more threads wait for resources held by each other and cannot continue.

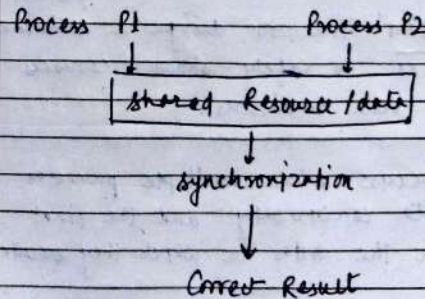
(iii) Starvation:- A thread waits for a very long time because other threads continuously get the required resources.

(iv) Synchronization Problems:- Improper synchronization can cause inconsistent or incorrect shared data.

Process Synchronization

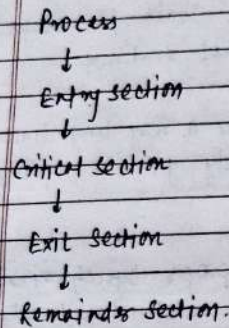
When multiple processes access the same resource simultaneously, incorrect results may occur.

Process synchronization is a mechanism used by the OS to coordinate multiple ~~resources~~ processes that share data or resources, so that they execute safely and produce correct results.



Critical section Problem

A critical section is a portion of a process where it access or modifies shared data or resources.



Entry section: The part where a process requests permission to enter the critical section.

Exit section: The part where a process releases the shared resource after leaving the critical section.

Remainder section: The part containing all other code that does not access shared resources.

Critical-section Problem: How to design a ^{protocol} ~~condition~~ that allows processes to safely share resources without causing race conditions.

Race Conditions: It occurs when multiple processes access shared data concurrently and the final result depends on the order in which they execute.

Requirements to solve critical-section Problem

A correct solution must satisfy three conditions:-

(i) Mutual Exclusion: Only one process can execute inside the critical section at a time.

* If P_1 is in critical section, P_2 must wait.

(ii) Progress: If no process is in critical section, the system should not unnecessarily prevent a waiting process from entering.

(iii) Bounded waiting: There must be a limit on how many times other processes can enter their critical sections after a process has requested entry.

* This prevents starvation.

Common Solutions

* Mutex locks

* Semaphores

* Monitors

* Peterson's Solution

* Hardware synchronization instructions.

Peterson's Solution

* It is a software based solution to the critical section problem for two processes.

* It ensures mutual exclusion, progress and bounded waiting (advantages)

It uses two shared variables:-

flag[i]:- indicates whether a process wants to enter critical section.

turn:- indicates whose turn it is to enter the critical section.

* It can have value 0 or 1

flag[i] = true (means process P_i wants to enter critical section)

flag[i] = false (means process P_i does not want to enter)

Peterson's Algorithm

for Process P_0

flag[0] = true;
turn = 1;

while (flag[1] == 1 && turn == 1)
; // wait

// critical section.

flag[0] = false;

// Remainder section.

for Process P_1

flag[1] = true;
turn = 0;

while (flag[0] == 1 && turn == 0)
; // wait

// critical section

flag[1] = false;

// Remainder Section.

P_0
↓
flag[0] = true
↓
turn = 1

P_1
↓
flag[1] = true
↓
turn = 0

Check conditions

One process enters critical section

Other process waits.

Working of Peterson's Solution:-

(i) P_0 sets its flag

(ii) P_0 gives turn to P_1

(iii) P_0 checks P_1 's status

(iv) P_0 enters critical section.

(v) P_0 leaves critical section.

(vi) P_1 can now enter.

Limitations:-

* Works only on two processes

* Uses busy waiting, which wastes CPU time while a process waits.

Synchronization Hardware

* Synchronization hardware refers to hardware-supplied instructions provided by the CPU that helps the operating system and programs safely coordinate multiple processes or threads accessing shared resources.

* When two processes access shared data simultaneously, incorrect results may occur. Hardware synchronization ensures that access to the shared data is properly controlled.

Main techniques:-

(i) Disabling Interrupts:- Temporarily prevents interruption while executing a critical section.

Disable Interrupts $\rightarrow$ Critical Section $\rightarrow$ Enable Interrupts.

(ii) Test and Set:- Automatically checks and sets a lock while (Test and set (lock))
; // next

// Critical section
lock = false;

(iii) Compare and Swap:- Compares a value and changes it only if the expected value matches.

Compare $\rightarrow$ if equal $\rightarrow$ swap.

(iv) Atomic Variables:- An atomic operation completes as a single, individual operation.

Atomic Increment $\rightarrow$ Read + Modify + Write $\rightarrow$ Cannot be interrupted midway.

Advantages:-

- * Provides fast synchronization.
- * Helps implement mutexes, semaphores and locks.
- * Prevents race conditions when used correctly.

Mutex Locks

A mutex lock (Mutual Exclusion lock) is a synchronization mechanism used to ensure that only one process or thread can access a shared resource at a time.

Working

Process
 $\downarrow$
Acquire lock
 $\downarrow$
Critical section
 $\downarrow$
Release Lock.

Ex:-

lock();
// critical section.
unlock();

Acquire:- Process obtains the lock before entering the critical section.

Locked:- other processes / threads must wait.

Release:- Lock is released after completing the critical section.

Important Properties of mutex

- (i) Mutual Exclusion
- (ii) Ownership
- (iii) Blocking
- (iv) Protection of shared data

Advantages

- * Prevents race conditions
- * Provides mutual exclusion.
- * Easy to understand and use.
- * Protects shared resources.

Semaphore

- * A semaphore is a synchronization mechanism used by an operating system to control access to shared resources and coordinate processes or threads.
- * Can control multiple resources.
- * Used for signaling as well as resource control.
- * Prevents race conditions.
- * Provides process / thread synchronization.

Basic operations

- (i) wait() / P(): Decreases the semaphore value.
- (ii) Signal() / V(): Indicates the semaphore value.
- * It indicates that a resource has been released

Types of semaphore

- (i) Binary semaphore: Value is generally 0 or 1.

* Used for mutual exclusion.

* $S=1 \rightarrow$ Resource available

$S=0 \rightarrow$ Resource unavailable

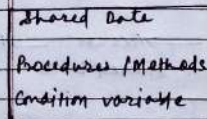
- (ii) Counting semaphore: Value can be 0, 1, 2, 3, ...

* Used when multiple identical resources are available.

Monitors in operating system

A monitor is a high level synchronization mechanism that allows processes or threads to safely share resources. It provides mutual exclusion, meaning only one process / thread can execute inside the monitor at a time.

Monitor



↓
only one thread enters at a time.

- Advantages: * Provides mutual exclusion.

* Easier to use than low level synchronization methods.

* Reduces programming errors.

Main Components of Monitor

(i) Shared Data:- contain data shared by multiple thread.
* Access to this data is controlled by monitor.

(ii) Procedures / Methods:- operations on the shared data are performed through procedures inside the monitor.

* Only one process can execute a monitor procedure at a time.

(iii) Condition Variables:- allow a process/thread to wait a particular condition and later be notified.

Common operations:-

wait():- makes a process wait until condition becomes true.
signal():- wakes a waiting process.

Working of a monitor

Process P1 $\rightarrow$ Enter Monitor $\rightarrow$ Execute procedure $\rightarrow$ Access shared Data $\rightarrow$ Exit Monitor

Process P2 $\rightarrow$ Wait $\rightarrow$ Enters after P1 exists.

Monitor v/s Semaphore

Monitor	Semaphore
* High-level synchronization mechanism	* Low-level synchronization mechanism
* Provides mutual exclusion automatically	* Use wait()/signal()
* Uses condition variables	* Use semaphore values.
* Easier to program	* More flexible but easier to misuse

Classic Problems of Synchronization

The classic problems of synchronization are standard problems used in OS to demonstrate how multiple processes or threads can safely share resources.

There are three major classic synchronization problems.

(i) Bounded-Buffer (Producer-Consumer) Problem.

(ii) Dining-Philosophers Problem

(iii) Readers-Writers Problem.

(i) Boundary-Buffer Problem:-

A producer produces data and puts it into a shared buffer while a consumer removes data from buffer.

* Synchronization Tools:- Semaphores and mutex locks.

Producer Challenge:-

Buffer

Consumer

* Producer does not add data when buffer is full.

* Consumer does not remove data when buffer is empty.

* Producer & consumer do not access the buffer simultaneously in an unsafe way.

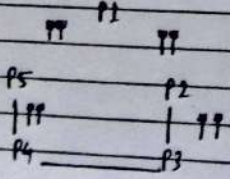
(ii) Dining-Philosophers Problem:-

* There are 5 philosophers sitting around a table.

* Each philosopher need two forks to eat.

* The problem is to prevent:- Deadlock
Starvation.

* Synchronization tools:- Semaphore
Mutex
Monitors.



(ii) Reader-Writer Problem:-

* This problem involves a shared resource such as database.

- Readers only read data
- Writers modify data.

* Multiple readers can read simultaneously but a writer needs exclusive access.

